

Projet : Développement d'un jeu

The logo for libGDX, featuring the word "lib" in black and "GDX" in red, set against a light gray rounded square background. Below the main logo, there is a faint, larger, and more stylized version of the same text.

Réalisé par :
Moustapha Ndiaye
Abdallah Remmide

Enseignant :
Morgane Vidal

Année universitaire 2025/2026

REMERCIEMENTS

Nous tenons tout d'abord à remercier notre professeur Madame Vidal pour son encadrement tout au long de ce module de programmation orientée objet. Les concepts abordés en cours, et la gestion de LibGDX, ont été fondamentaux pour la réalisation de ce projet. Ce travail nous a permis de mettre en pratique la théorie et de comprendre les enjeux du développement d'une application complète en Java.

RÉSUMÉ

Circuit Laser est un jeu de réflexion et d'adresse développé en Java avec la bibliothèque LibGDX. Le but du jeu est de guider un faisceau laser à travers différents niveaux labyrinthiques, du point de départ jusqu'à la sortie, sans toucher les murs ni les obstacles. Le projet met en œuvre la gestion de collision, un système de niveaux et une caméra adaptative pour garantir une expérience visuelle propre quelle que soit la taille de la fenêtre.

SOMMAIRE

1. INTRODUCTION	3
2. DÉVELOPPEMENT ET ARCHITECTURE TECHNIQUE	3
2.1 Choix technologiques et structure	3
2.2 Conception du plateau et des niveaux	4
2.3 Gestion du Laser et des collisions	4
2.4 Interface et Expérience Utilisateur	4
3. CONCLUSION	5

1. INTRODUCTION

Dans le cadre du module R3-10, il nous a été demandé de concevoir et développer une application graphique interactive en utilisant le langage Java. L'objectif était de maîtriser les concepts de la Programmation Orientée Objet (POO) tout en découvrant l'utilisation d'un framework graphique externe (LibGDX). Nous avons choisi de réaliser "Circuit Laser", un jeu d'arcade qui demande au joueur de la précision et de la réflexion. Ce choix a été motivé par la volonté de travailler sur des problématiques classiques du jeu vidéo, telles que le déplacement vectoriel, la détection de collisions sur grille et la gestion des états de jeu. Ce rapport détaille les étapes de conception, les choix techniques effectués, ainsi que les solutions apportées aux problèmes rencontrés durant le développement.

2. DÉVELOPPEMENT ET ARCHITECTURE TECHNIQUE

2.1 Choix technologiques et structure

Pour mener à bien ce projet, nous avons utilisé le framework LibGDX. Ce choix s'est imposé naturellement car nous avons commencé à l'utiliser lors des séances de Travaux Pratiques. Cette familiarisation préalable avec l'outil nous a permis de nous concentrer rapidement sur la logique du jeu plutôt que sur l'apprentissage technique d'un nouveau moteur.

L'architecture du projet a été pensée pour respecter les principes de la POO. Le code est réparti en plusieurs classes distinctes ayant chacune une responsabilité unique. La classe principale "Main" agit comme le chef d'orchestre. Elle initialise les ressources graphiques, gère la boucle de rendu principale et contrôle les différents états du jeu, comme le menu, la partie en cours ou l'écran de victoire. Cette centralisation facilite la maintenance et l'évolution du code.

2.2 Conception du plateau et des niveaux

La gestion des niveaux repose sur une approche matricielle. La classe "Plateau" contient la logique du monde, représentée par des tableaux à deux dimensions d'entiers (`int[][]`).

Chaque chiffre dans ces tableaux correspond à un type de terrain : un mur, un chemin, le point de départ ou l'arrivée (1, 2, 3 ou 4). Cette abstraction permet de créer et modifier des niveaux très facilement sans toucher au code du moteur de jeu. Nous avons conçu trois niveaux de jeu plutôt simples mais ludiques. L'objectif était de privilégier le plaisir de jeu ("fun") avec des parcours accessibles et divertissants. Pour l'affichage, le jeu dissocie la logique (la grille) du rendu visuel (les textures), ce qui permet d'avoir un code plus propre et modulable.

2.3 Gestion du Laser et des collisions

Le personnage principal, le "Laser", est géré par sa propre classe. Son déplacement n'est pas case par case, mais fluide, calculé en temps réel grâce au temps écoulé entre chaque image (le delta time). Cela rend le jeu plus agréable et réactif. La partie la plus technique a été la gestion des collisions. À chaque déplacement, le programme calcule la position future du laser et la convertit en coordonnées de grille pour vérifier le contenu de la case correspondante dans le tableau du niveau. Si la case est un mur, l'état du jeu bascule en "Game Over". Nous avons également implémenté un système de "traînée" visuelle (ArrayList de vecteurs) qui dessine le chemin parcouru par le laser, ajoutant un retour visuel important pour le joueur.

2.4 Interface et Expérience Utilisateur

Un point important du développement a été la gestion de la caméra et de l'affichage. Au début du projet, le redimensionnement de la fenêtre déformait les graphismes. Pour pallier ce problème, nous avons mis en place une OrthographicCamera couplée à un FitViewport. Ce système garantit que le jeu conserve ses proportions et reste net sur n'importe quel écran. Enfin, l'interface utilisateur (UI) guide le joueur à travers les différents états : un écran d'accueil, des messages de victoire ou de défaite, et une transition fluide entre les niveaux, assurant une boucle de jeu complète et infinie une fois le jeu terminé.

3. CONCLUSION

La réalisation de Circuit Laser a été une expérience très enrichissante qui nous a permis de consolider nos compétences en Java. Nous avons pu constater l'importance d'une bonne architecture logicielle avant de commencer à coder. La séparation entre les données (le plateau) et l'affichage a grandement simplifié l'ajout de nouveaux niveaux. Nous avons rencontré quelques difficultés, notamment pour ajuster la précision des collisions et gérer correctement l'inversion de l'axe Y entre le rendu graphique et le tableau de données, mais ces problèmes ont été résolus par la suite. Si nous devions améliorer ce projet à l'avenir, nous ajouterions des niveaux de difficultés plus élevés. Également rajouter un éditeur de niveaux intégré pour permettre aux joueurs de créer leurs propres cartes. Ainsi, ce projet constitue une base solide pour de futurs développements plus complexes.